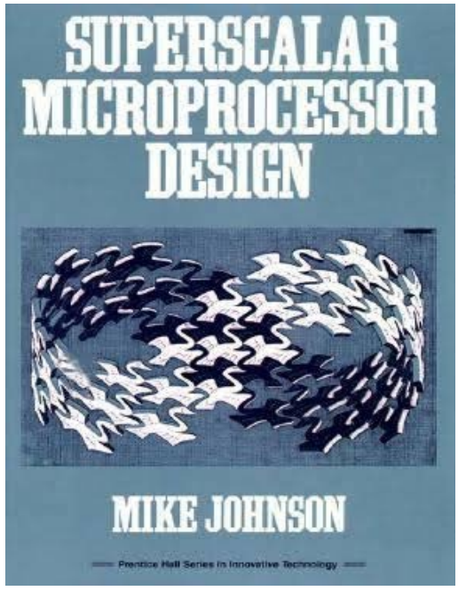
https://www.amazon.com/Superscalar-Microprocessors-Design-Mike-Johnson/dp/0138756341

18-740: Modern Computer Architecture & Design

Lecture 2:
"Modern Superscalar Out-of-Order Processors"

John Paul Shen
August 27, 2026



- **Lab 0** posted: start working on it; bring your questions to recitation
  - Lab 0 tasks are due on Monday, August 31, 2026 (11:59pm ET)
- Look for team partners for the team lab assignments (**Labs 1-4**)
- **Required Reading Assignment**:
  - **Read Chapters 2 and 4 of Shen and Lipasti (SnL)**

Electrical & Computer ENGINEERING

18-740: Modern Computer Architecture & Design

Lecture 2:
"Modern Superscalar Out-of-Order Processors"

A. Modern Superscalar Processor Organization
   - ➢ Exploit Instruction Level Parallelism (ILP)

B. Instruction Flow Techniques
   1. Control Flow Graph and Branch Instructions
   2. Dynamic Branch Prediction Tasks
      a. Branch Target Prediction
      b. Branch Direction Prediction

Electrical & Computer ENGINEERING

18-740: Modern Computer Architecture & Design

Evolution of Von Neumann Architecture Implementations:

- ~~SP: Sequential Processors (direct implementation of sequential execution)~~
- PP: Pipelined Processors (overlapped execution of in-order instructions)
- SSP: Superscalar Processors (out-of-order execution of multiple instructions)
- MCP: Multi-core Processors (concurrent multi-threads and/or multi-programs)
- HMP: Heterogeneous Multi-Processors (concurrent multi-tasks on diverse co-processors)

18-740

Parallelisms for **Performance** (→ for **Power** Reduction → for **Energy** Efficiency)

- **ILP**: Basic Block (exploit ILP in PP, SSP)
- **ILP**: Loop Iteration (exploit ILP in SSP, VLIW)
- **DLP**: Data Set (exploit DLP in VP/SIMD, GPU)
- **TLP**: Thread/Task (exploit TLP in MCP, HMP)
- **SLP**: System/Workload (exploit SLP in Multi-Computer Clusters)

18-740



"Iron Law" of Processor Performance

$$1/\text{ComputerPerformance} = \frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

(inst. count) (CPI) (cycle time)

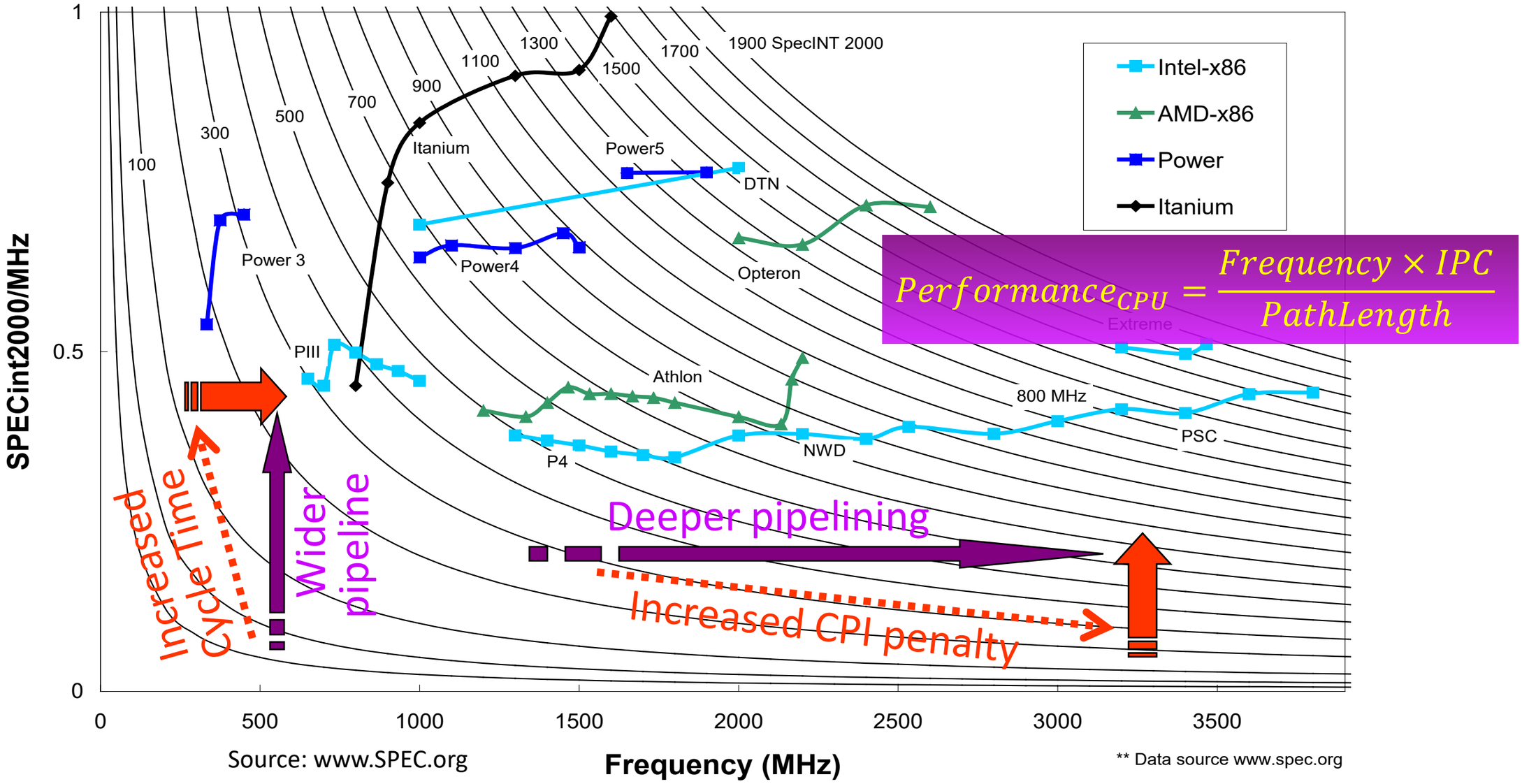
Architecture → Implementation → Realization

Compiler Designer Processor Designer Chip Designer

- In the 1980's: **(pipelining)**
  - CPI: 5.0 → 1.2
- In the 1990's: **(superscalar)**
  - CPI: 1.2 → 0.5
  - IPC: 0.8 → 2.0
- In the 2000's: **(multicores)**
  - Hit power wall !!!
  - ILP → TLP

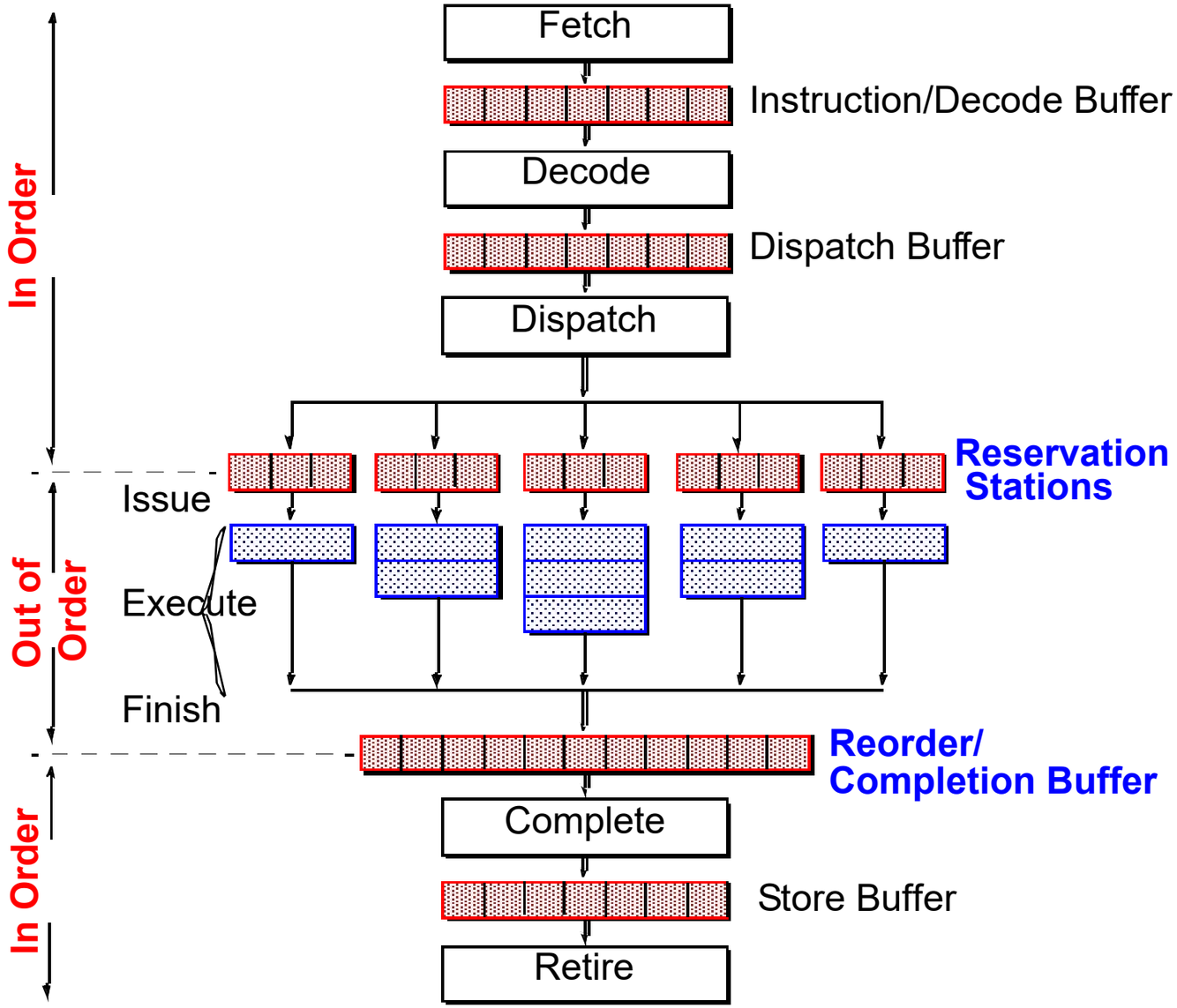
[John DeVale & Bryan Black, 2005]

Landscape of Processor Families [SPECint2000]



$$Performance_{CPU} = \frac{Frequency \times IPC}{PathLength}$$

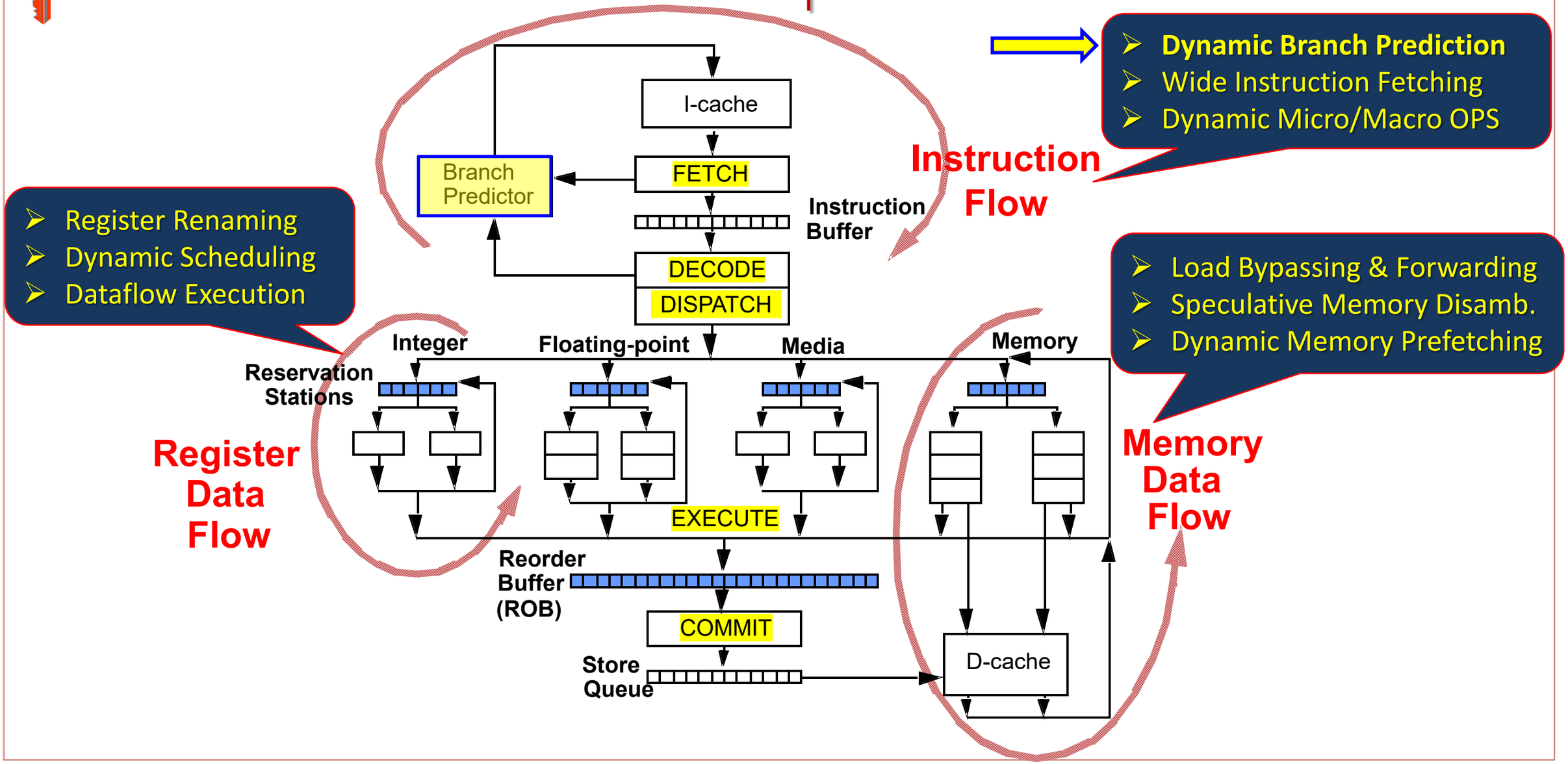
A. Modern Superscalar Processor Organization



- Buffers provide decoupling
- In OOO designs they also facilitate (re-)ordering
- More details on specific buffers to follow

[Mike Johnson, *Superscalar Microprocessor Design*, Prentice-Hall, 1991]

Three Flow Paths of Superscalar Processors



18-740: Modern Computer Architecture & Design

Lecture 2: "Modern Superscalar Out-of-Order Processors"

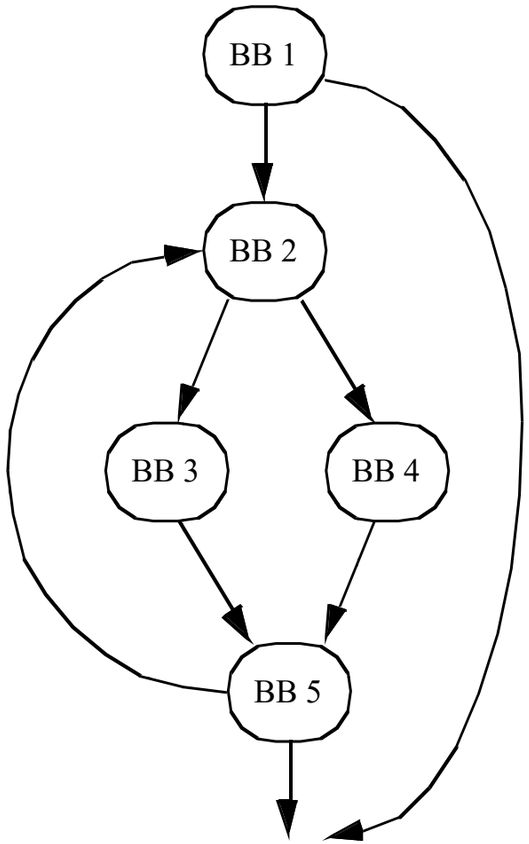
A. Modern Superscalar Processor Organization
   - Exploit Instruction Level Parallelism (ILP)

B. Instruction Flow Techniques
   1. Control Flow Graph and Branch Instructions
   2. Dynamic Branch Prediction Tasks
      a. Branch Target Prediction
      b. Branch Direction Prediction

Electrical & Computer ENGINEERING

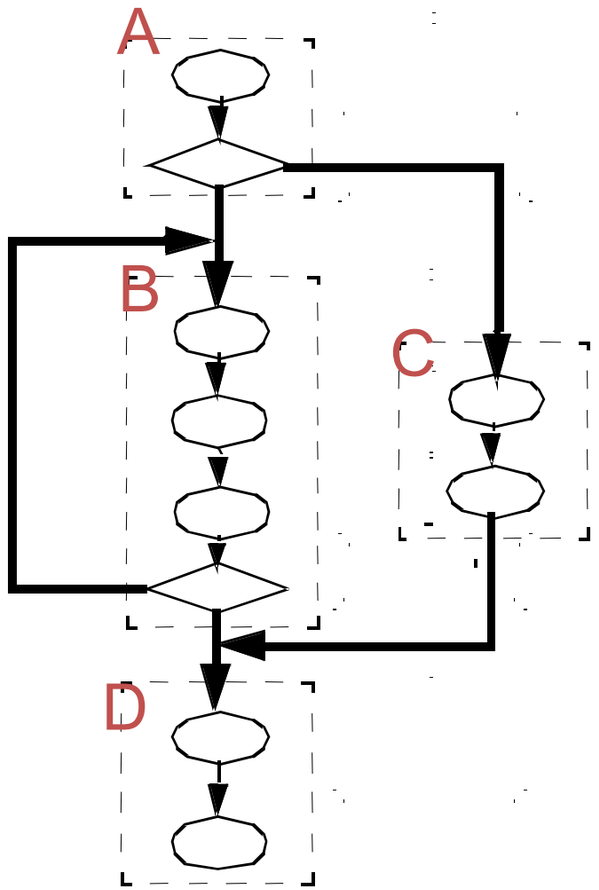
B.1. Control Flow Graph and Branch Instructions

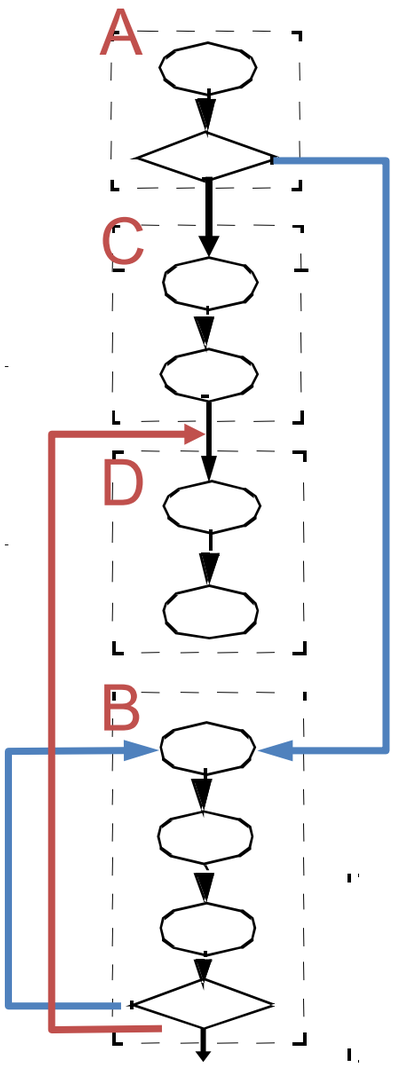
- Your program is actually a control flow graph
  - Shows possible paths of control flow through basic blocks
- Control Dependence
  - Node *X* is control dependent on Node *Y* if the computation in *Y* determines whether *X* executes

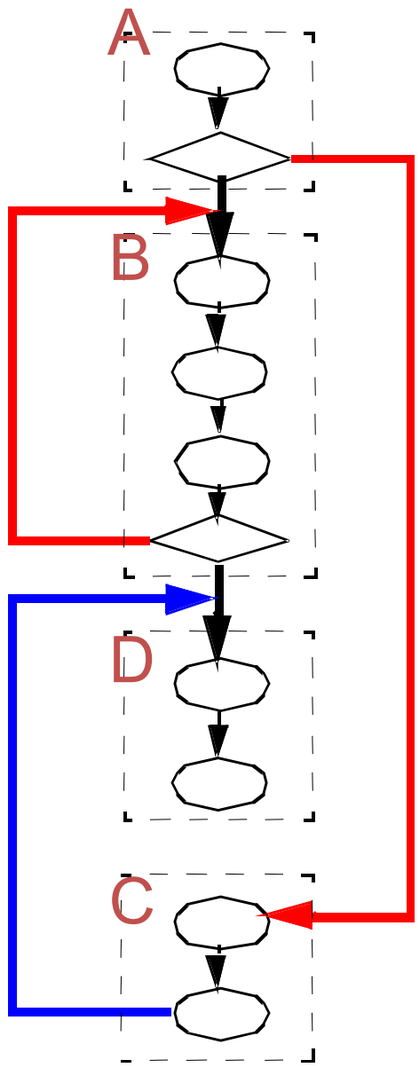



```
main:
        addi  r2, r0, A          |
        addi  r3, r0, B          |
        addi  r4, r0, C          |   BB 1
        addi  r5, r0, N          |
        add   r10,r0, r0         |
        bge   r10,r5, end        |
loop:                            |
        lw    r20, 0(r2)         |
        lw    r21, 0(r3)         |   BB 2
        bge   r20,r21,T1         |
        sw    r21, 0(r4)         |   BB 3
        b     T2                 |
T1:                              |
        sw    r20, 0(r4)         |   BB 4
T2:                              |
        addi  r10,r10,1          |
        addi  r2, r2, 4          |
        addi  r3, r3, 4          |   BB 5
        addi  r4, r4, 4          |
        blt   r10,r5, loop       |
end:
```

Mapping CFG to Linear Instruction Sequence







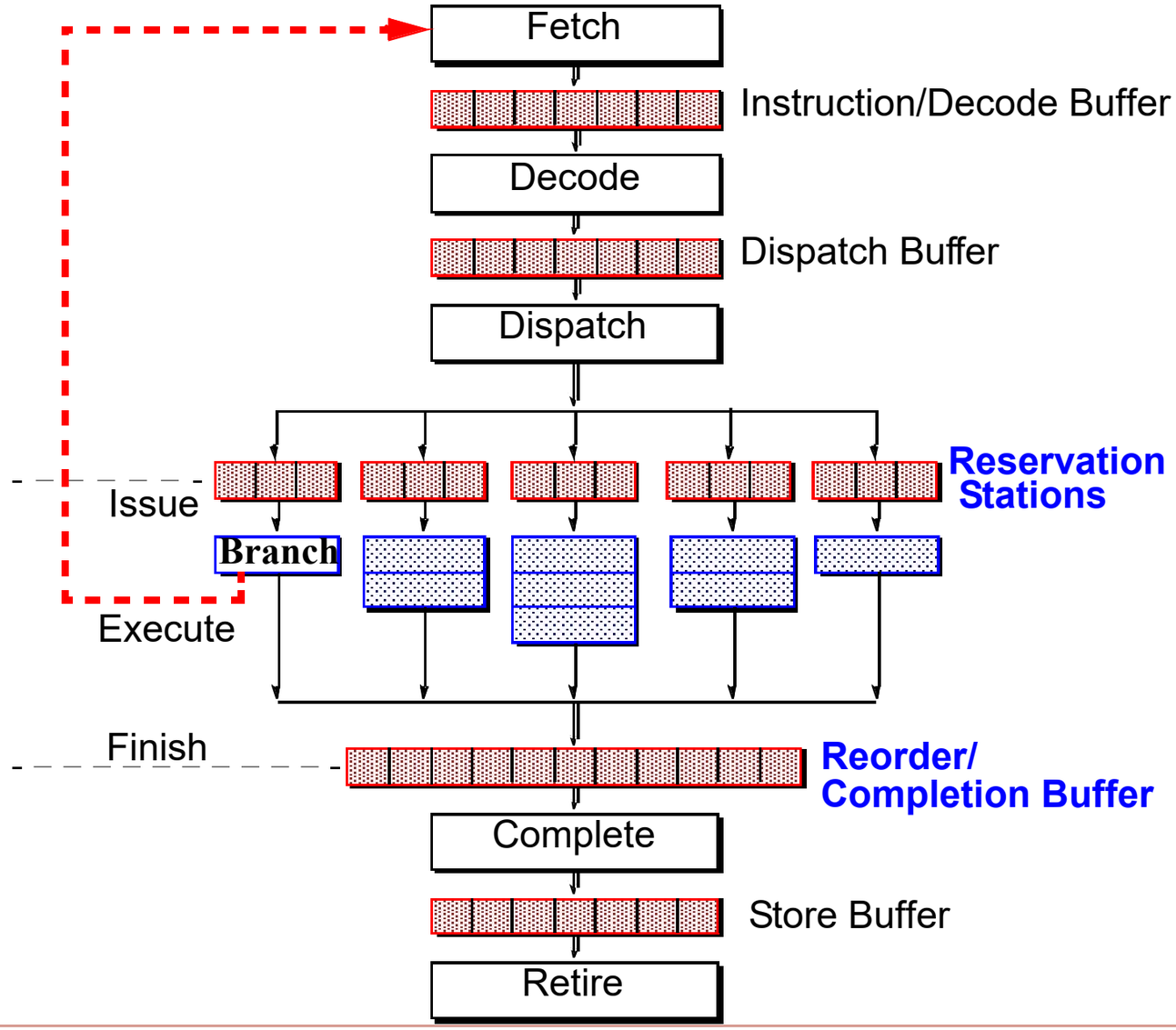
Branch Types and Implementation

- Types of Branches
  - Conditional or Unconditional?
  - Subroutine Call (aka Link), needs to save PC?
  - How is the branch target computed?
    - Static Target e.g. immediate, PC-relative
    - Dynamic targets e.g. register indirect
- Conditional Branch Architectures
  - Condition Code "*N-Z-C-V*" *e.g. PowerPC*
  - General Purpose Register *e.g. Alpha, MIPS*
  - Special Purposes register *e.g. Power's Loop Count*

What's So Bad About Branches?

- Robs instruction fetch bandwidth and ILP
  - Use up execution resources
  - Fragmentation of I-cache lines
  - Disruption of sequential control flow
    - Need to determine branch direction (conditional branches)
    - Need to determine branch target
- Example:
  - We have a N-way superscalar processor (N is large)
  - A branch every 5 instructions that takes 3 cycles to resolve
  - What is the effective fetch bandwidth?

Disruption of Sequential Control Flow



https://ieeexplore.ieee.org/document/1672107

Riseman and Foster's Study

- 7 benchmark programs on CDC-3600
- Assume infinite machine:
  - Infinite memory and instruction stack, register file, fxn units

    Consider only true dependency at data-flow limit
- If bounded to single basic block, i.e. no bypassing of branches $\Rightarrow$ maximum speedup is **1.72**
- Suppose one can bypass conditional branches and jumps (i.e. assume the actual branch path is always known such that branches do not impede instruction execution)

Br. Bypassed:	*0*	*1*	*2*	*8*	*32*	*128*
Max Speedup:	***1.72***	***2.72***	***3.62***	***7.21***	***24.4***	***51.2***

B.2. Dynamic Branch Prediction Tasks

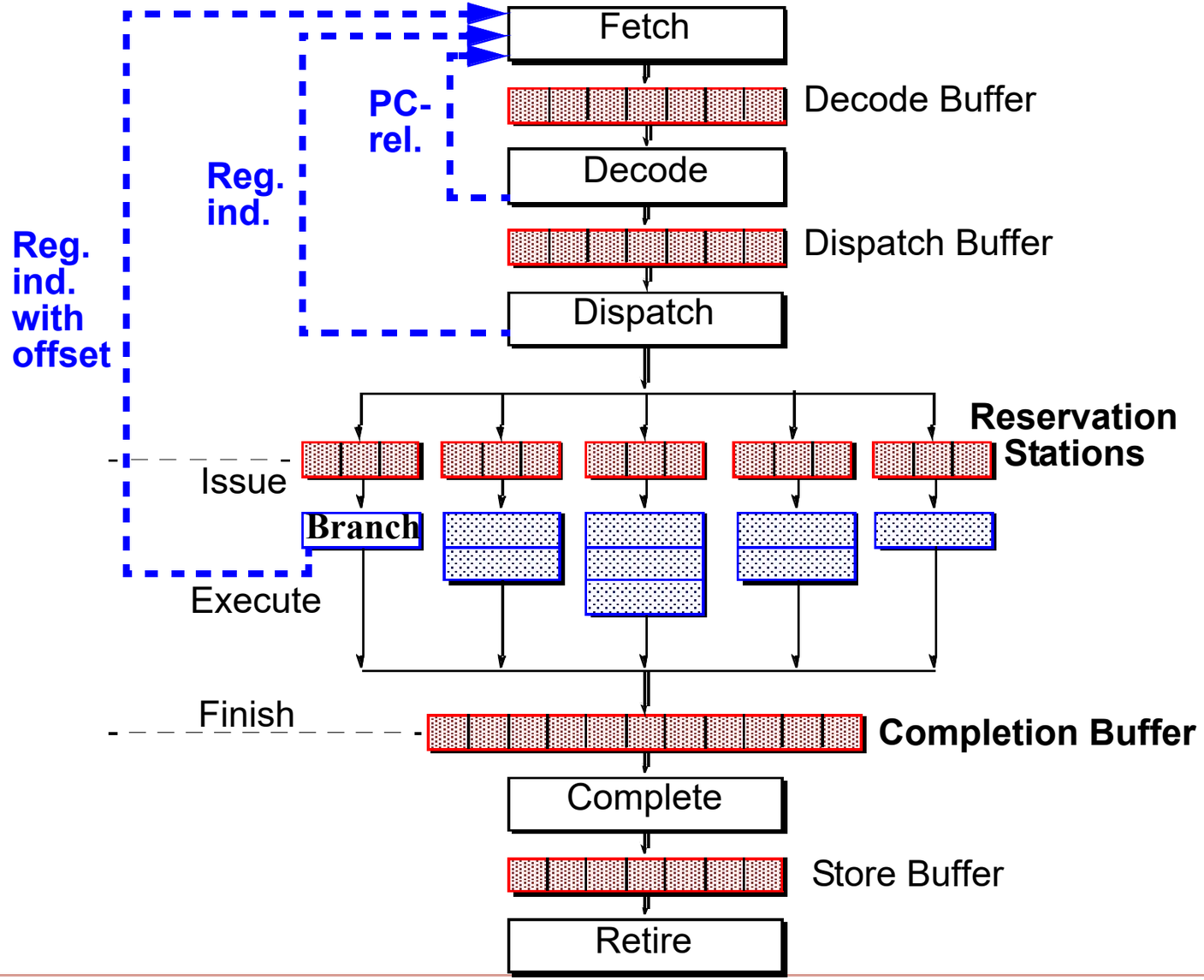
- Target Address Generation
  - Access register
    - PC, GP register, Link register
  - Perform calculation
    - +/- offset, auto incrementing/decrementing

  ⇒ Target Speculation

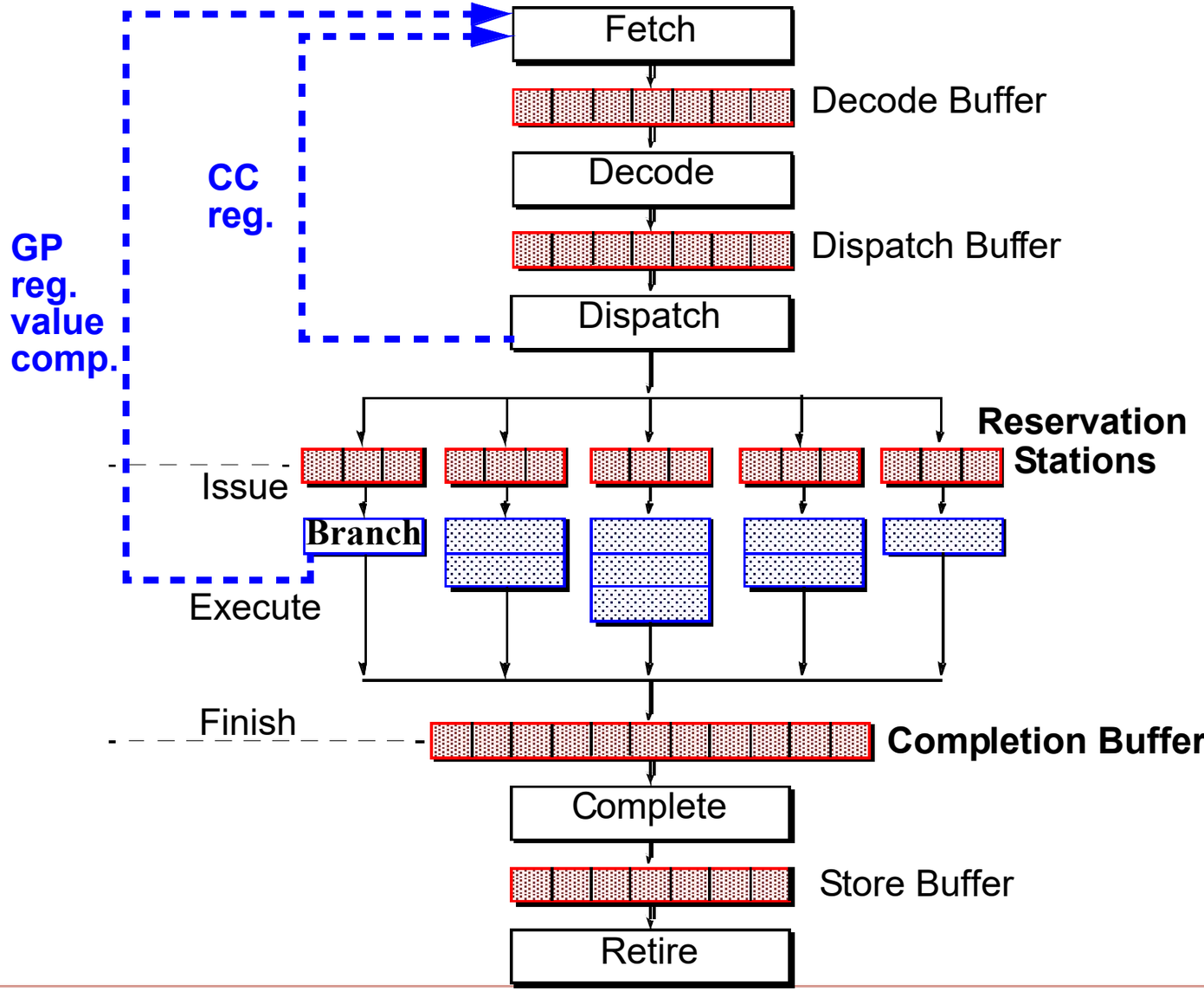
- Condition Resolution
  - Access register
    - Condition code register, data register, count register
  - Perform calculation
    - Comparison of data register(s)

  ⇒ Condition Speculation

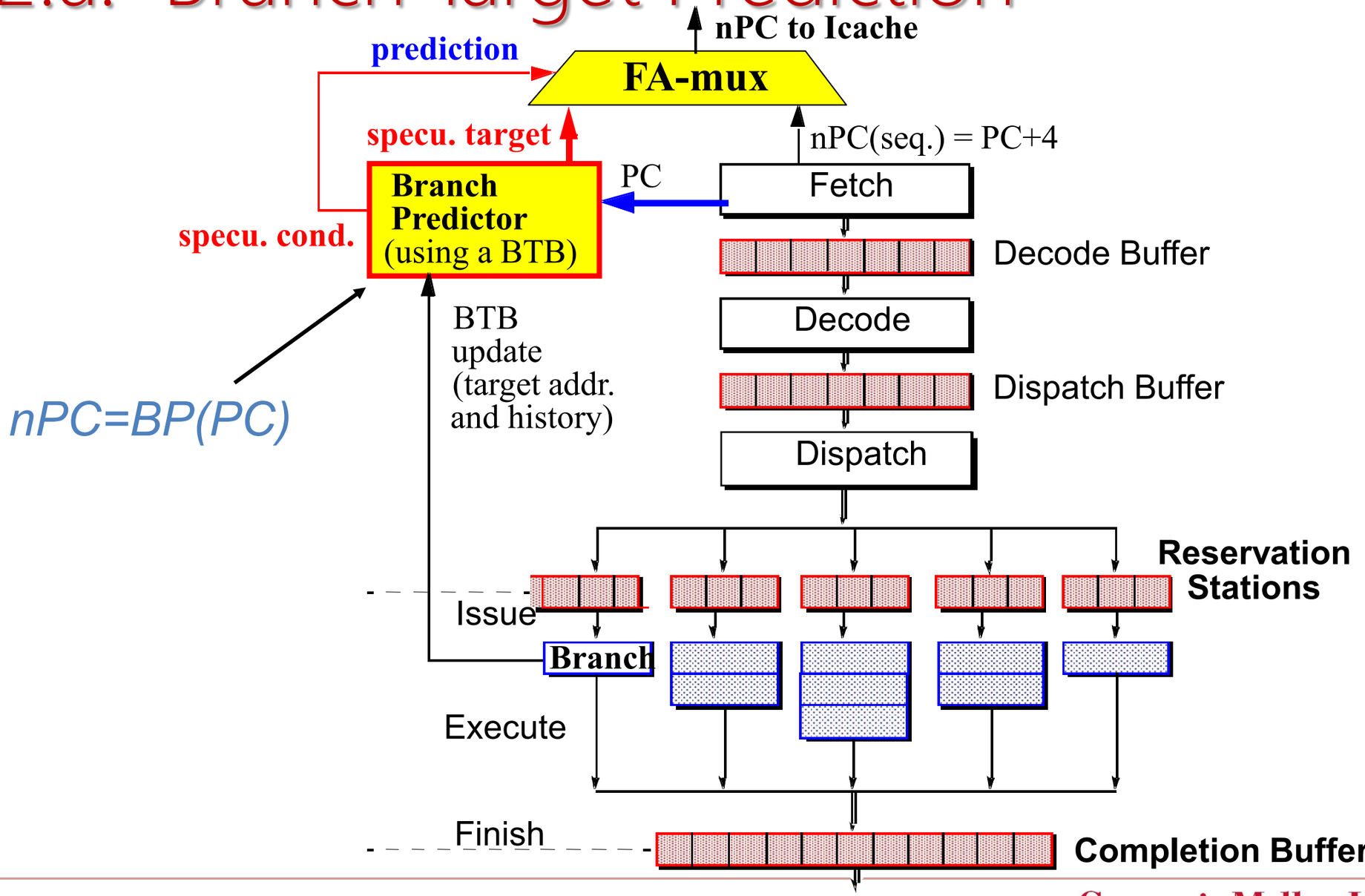
Target Address Generation



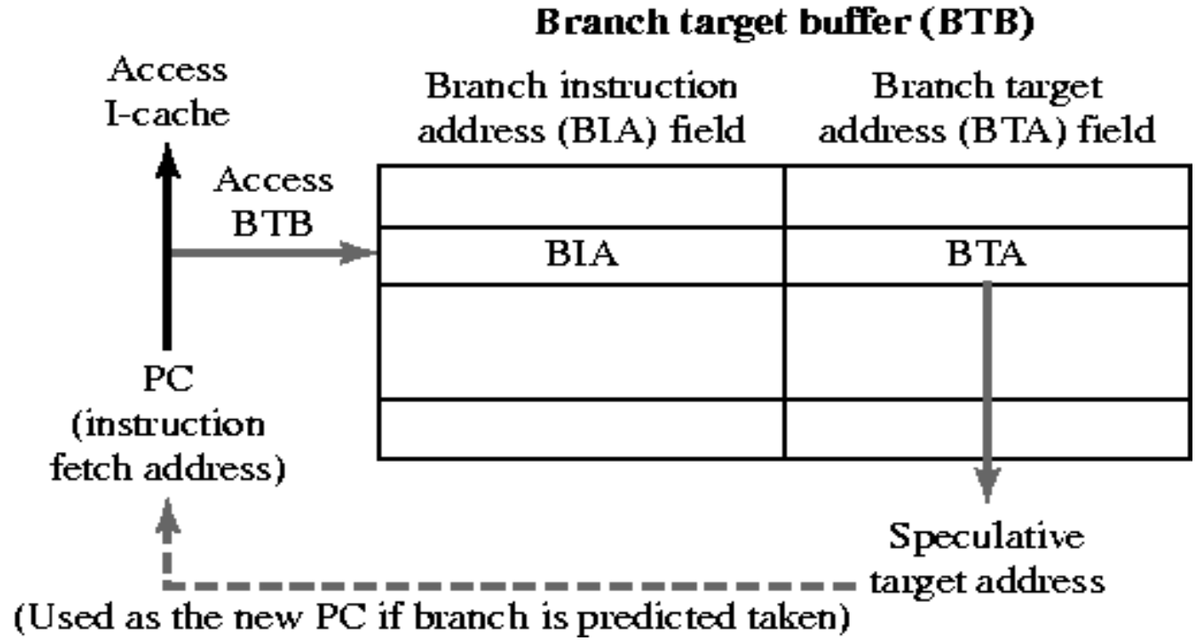
Condition Resolution

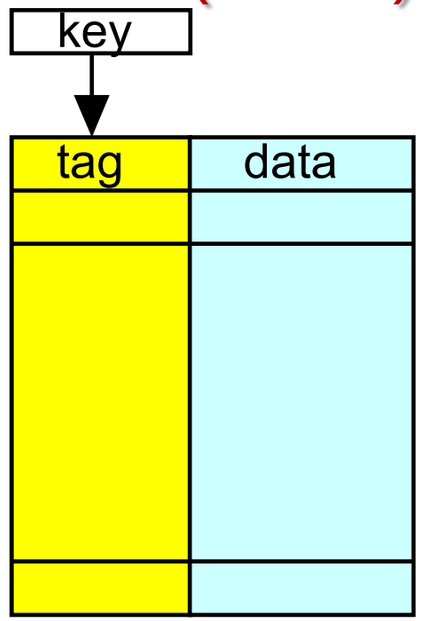


B.2.a. Branch Target Prediction



Target Prediction: Branch Target Buffer (BTB)



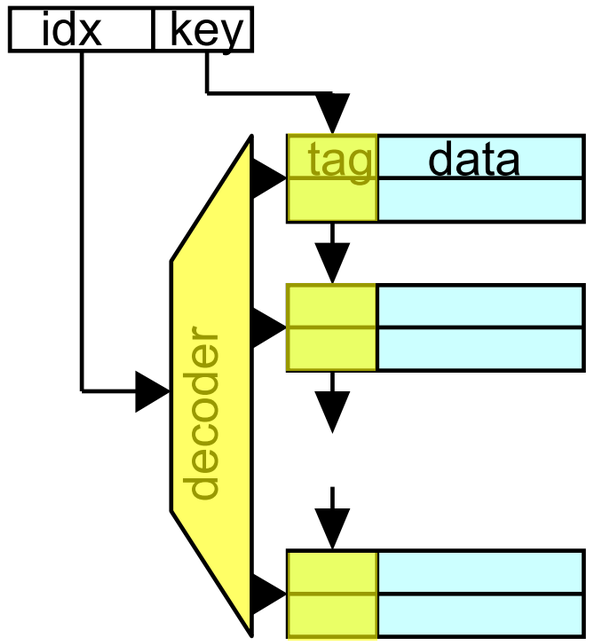


Associative Memory (CAM)

- A small "cache-like" memory in the instruction fetch stage
- Remembers previously executed branches, their addresses (PC), information to aid target prediction, and most recent target addresses
- I-fetch stage compares current PC against those in BTB to "guess" nPC
  - If matched then prediction is made else nPC=PC+N
  - If predict taken then nPC=target address in BTB else nPC=PC+N
- When branch is actually resolved, BTB is updated

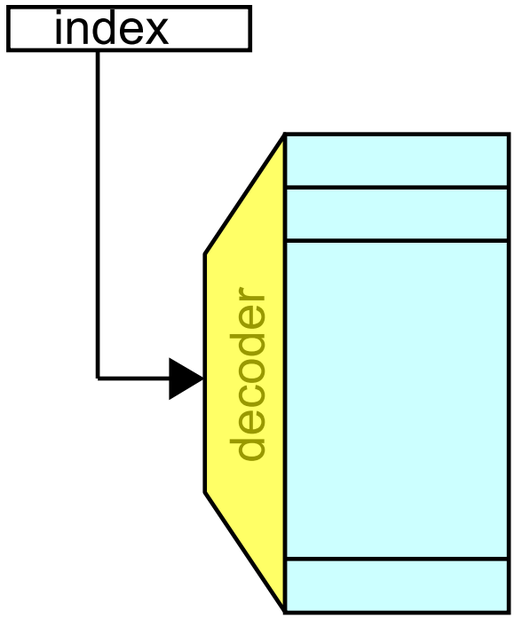
More on BTB (aka BTAC)

- Typically, a large associative structure
  - Pentium3: 512 entries, 4-way; Opteron: 2K entries, 4-way
- Entry format
  - Valid bit, address tag (PC), target address, fall-through BB address (length of BB), branch type info, branch direction prediction
- BTB provides both target and direction prediction
- Multi-cycle BTB access?
  - The case in many modern processors (2 cycle BTB)
  - Start BTB access along with I-cache in cycle 0
  - In cycle 1, fetch from PC+N (predict not-taken)
  - In cycle 2, use BTB output to verify
    - 1 cycle fetch bubble if branch was taken



N-Way
Set-Associative Memory
k-bit index
2^k • N blocks

Memory (RAM) Implementation Options



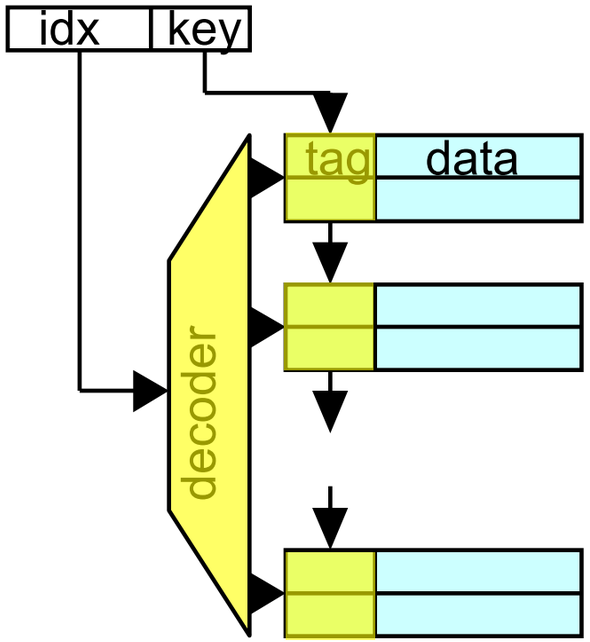
Indexed Memory

k-bit index
2^k blocks



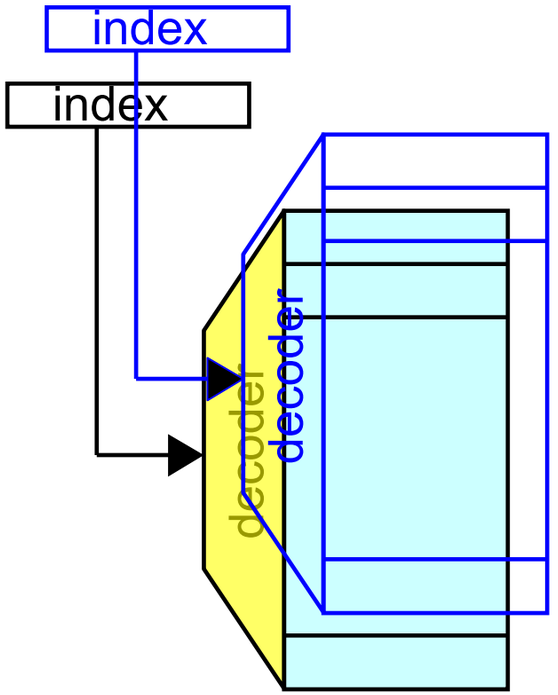
Associative Memory (CAM)

no index
unlimited blocks



N-Way Set-Associative Memory

k-bit index
2^k • N blocks



Indexed Memory (Multi-Ported)

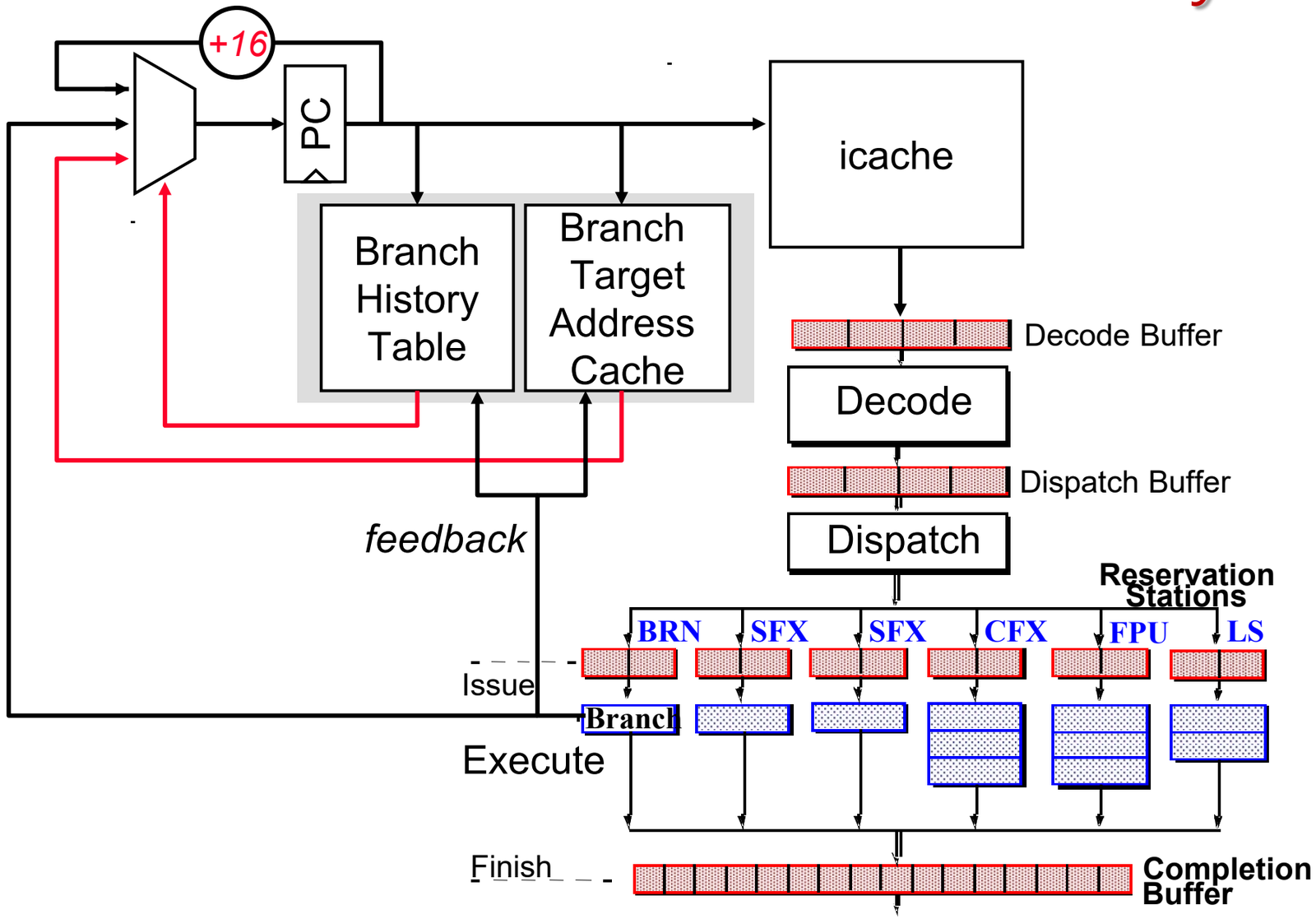
(2x) k-bit index
(2x) 2^k blocks

B.2.b. Branch Direction Prediction

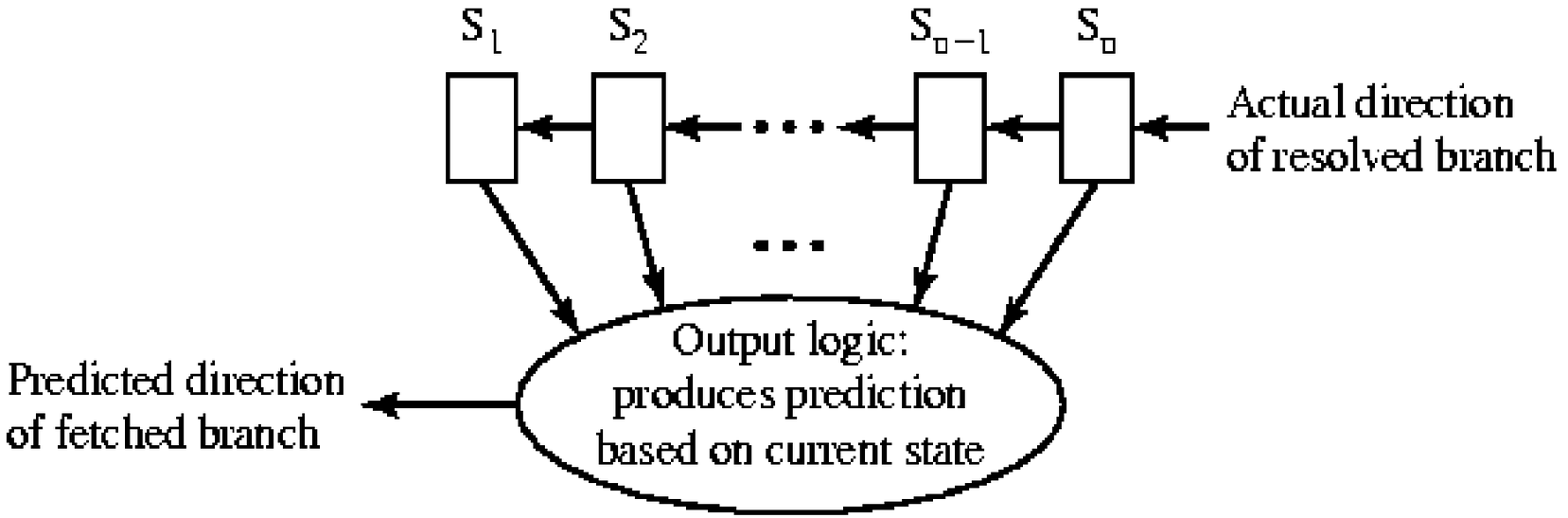
- Biased For Not Taken
  - Does not affect the instruction set architecture; Not effective in loops
- Software Prediction
  - Encode an extra bit in the branch instruction
    - Predict not taken: set bit to 0
    - Predict taken: set bit to 1
  - Bit set by compiler or user; can use profiling
  - Static prediction, same behavior every time
- Prediction Based on Branch Offsets
  - Positive offset: predict not taken
  - Negative offset: predict taken
- Prediction Based on History

Branch Direction Prediction Based on History

- Use HW tables to track history of direction/targets
  - nextPC = function(PC, history)
- Need to verify prediction
  - Branch still gets to execute



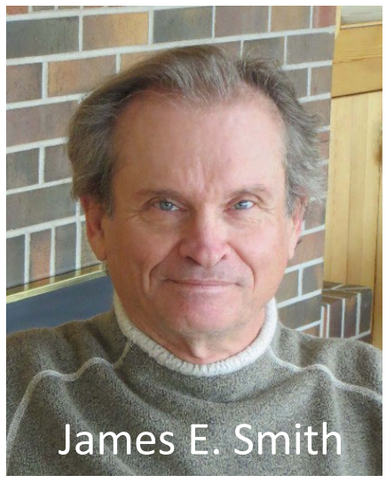
History-Based Branch Direction Prediction

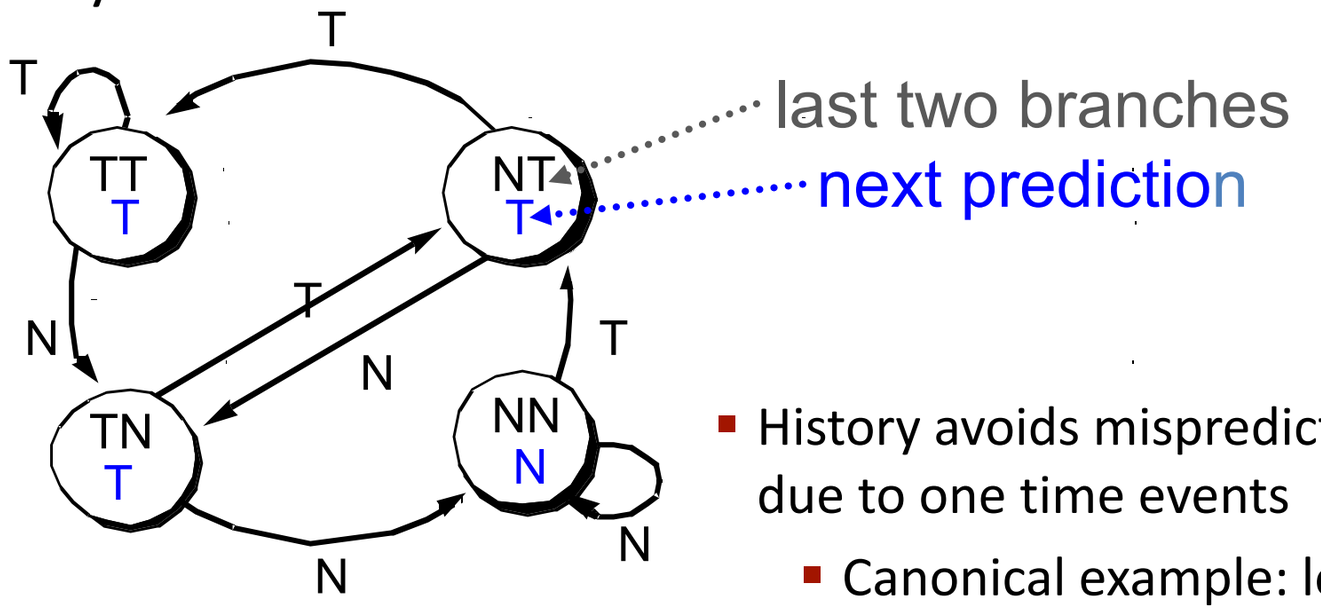


- Track history of previous directions of branches (T or NT)
- History can be local (per static branch) or global (all branches)
- Based on observed history bits (T or NT), a FSM makes a prediction of Taken or Not Taken
- Assumes that future branching behavior is predictable based on historical branching behavior

Example Prediction Algorithm

- Prediction accuracy approaches maximum with as few as 2 preceding branch occurrences used as history



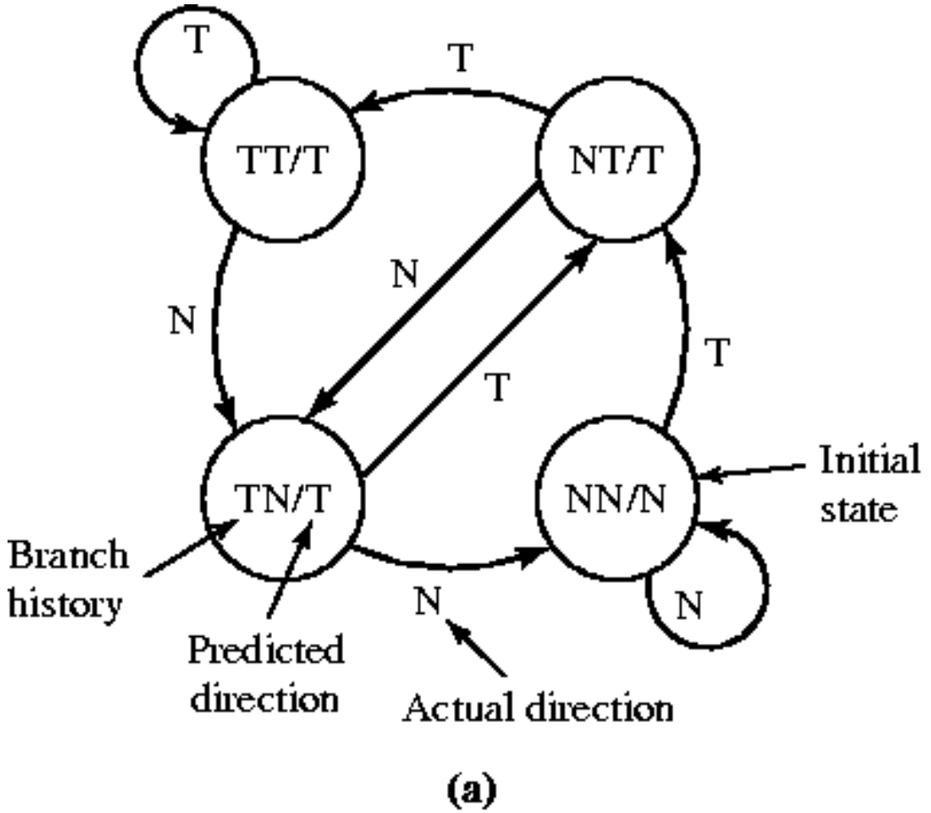


Results (%)

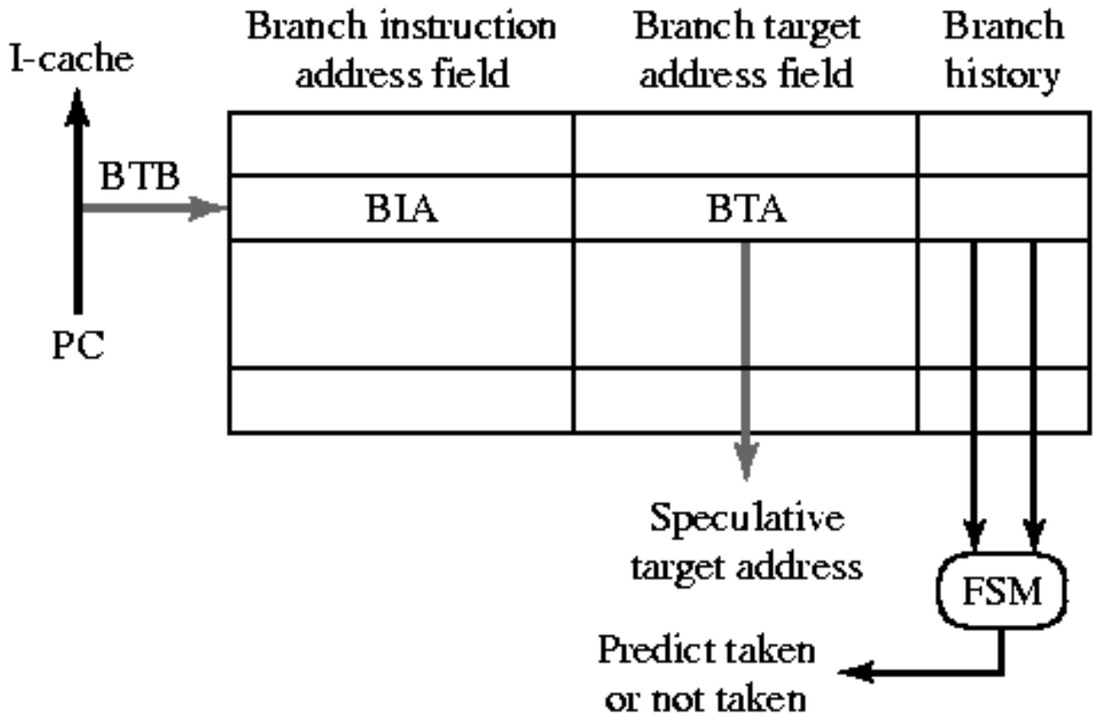
IBM1	IBM2	IBM3	IBM4	DEC	CDC
93.3	96.5	90.8	83.4	97.5	90.6

- History avoids mispredictions due to one time events
  - Canonical example: loop exit
- 2-bit FSM as good as n-bit FSM
- Saturating counter as good as any FSM

History-Based Branch Prediction

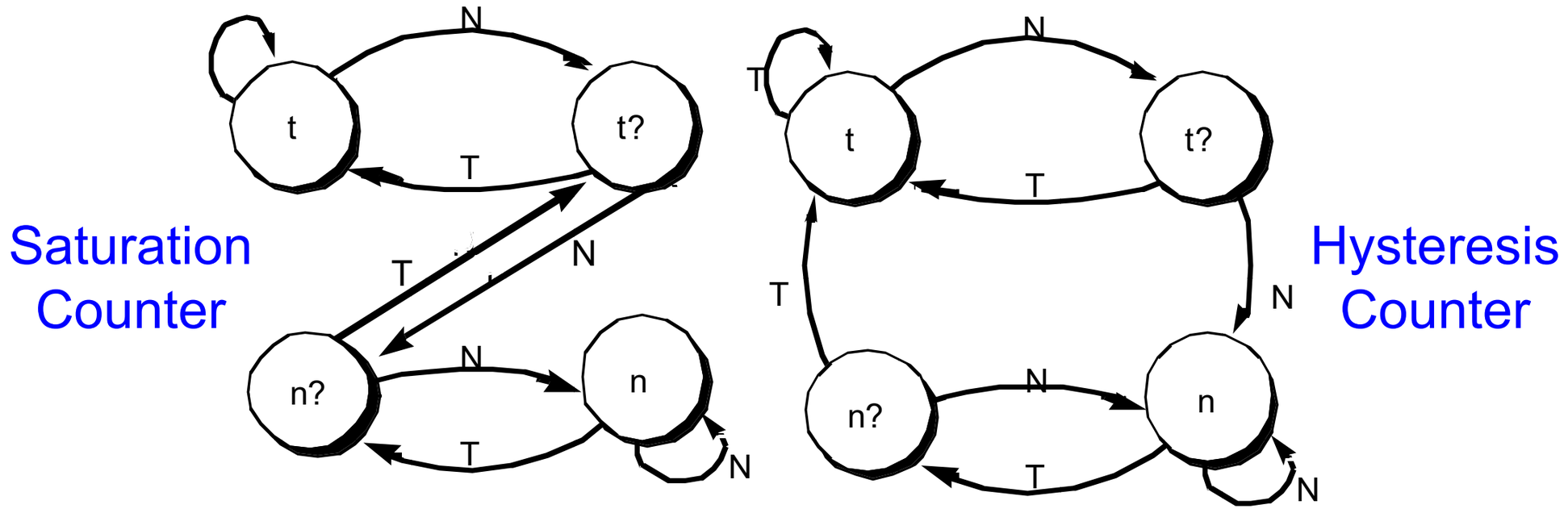


(a)



(b)

Other Prediction Algorithms



- Combining prediction accuracy with BTB hit rate (86.5% for 128 sets of 4 entries each), branch prediction can provide the net prediction accuracy of approximately 80%. This implies a 5-20% performance enhancement.

[Bryan Black, Motorola, 1994]

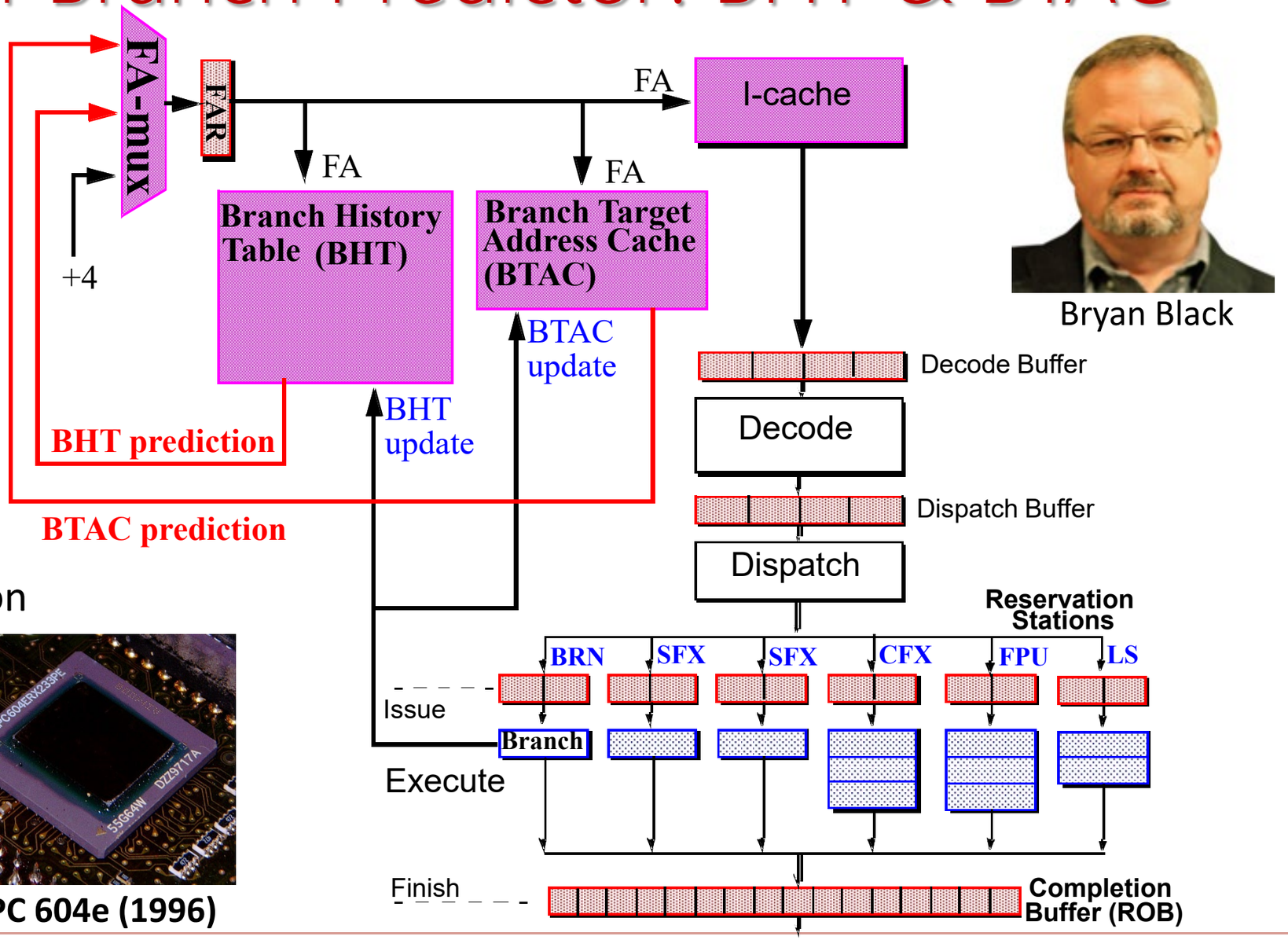
PowerPC 604 Branch Predictor: BHT & BTAC

BTAC:

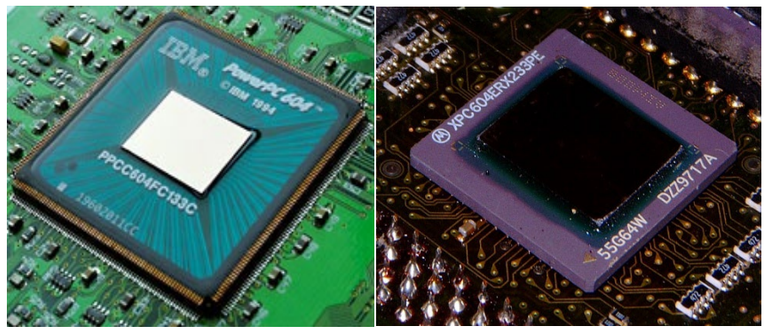
- 64 entries
- Fully associative
- Hit → predict taken

BHT:

- 512 entries
- Direct mapped
- 2-bit saturating counter
- History based prediction
- Overrides BTAC prediction



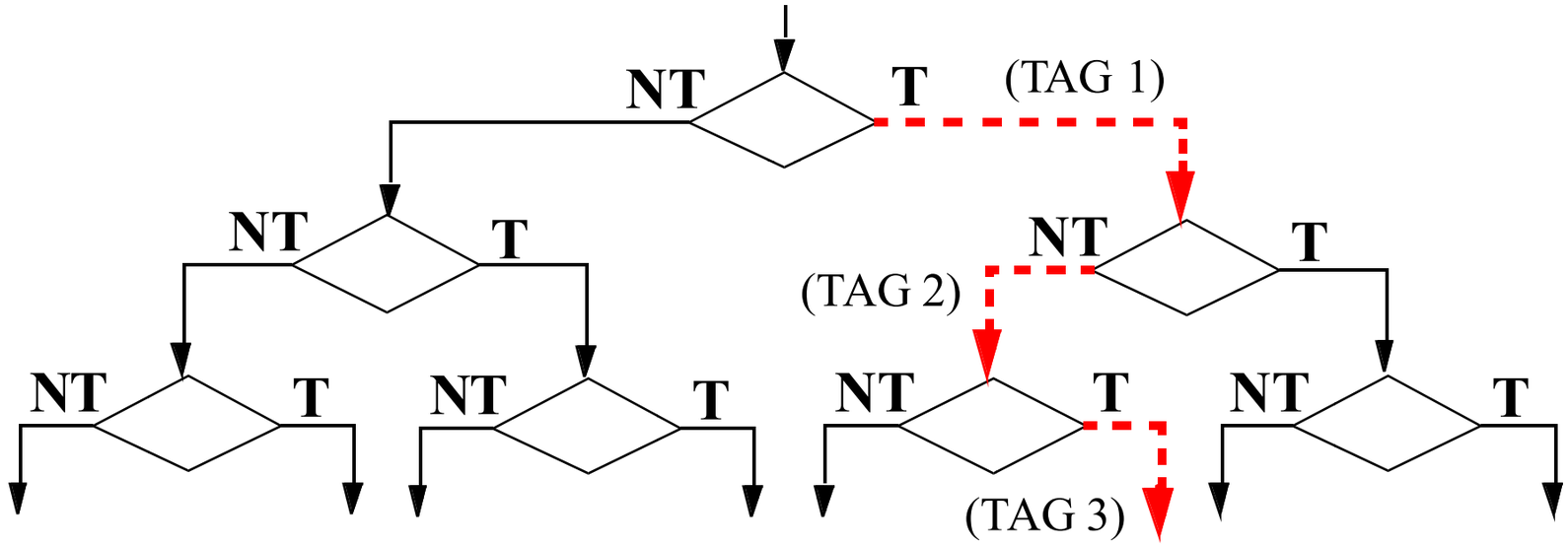
Bryan Black



4-IPC 604 (1994)

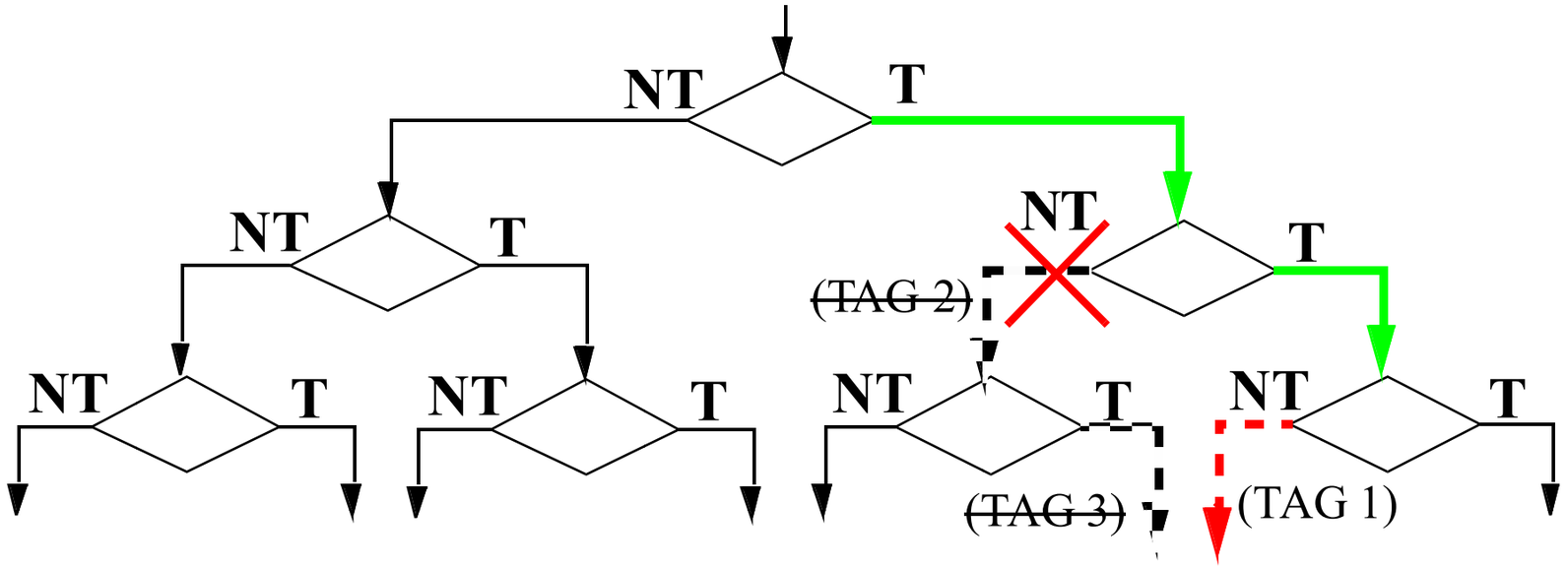
6-IPC 604e (1996)

Control Flow Speculation



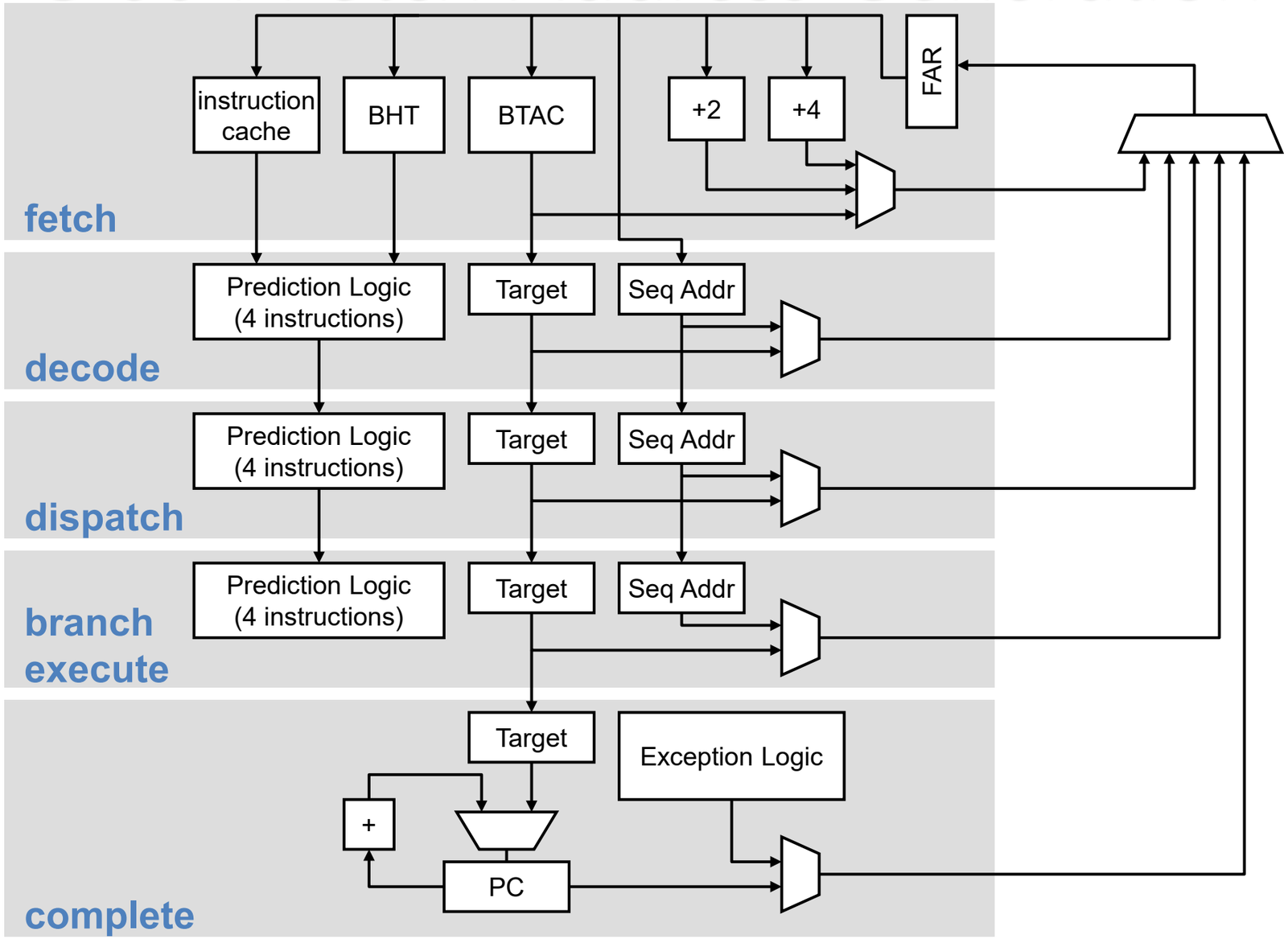
- Leading Speculation
  - Typically done during the Fetch stage
  - Based on potential branch instruction(s) in the current fetch group
- Trailing Confirmation
  - Typically done during the Branch Execute stage
  - Based on the next Branch instruction to finish execution

Mis-speculation Recovery



- Start New Correct Path
  - Must remember the alternate (non-predicted) path
- Eliminate Incorrect Path
  - Must ensure that the mis-speculated instructions produce no side effects

PowerPC 604 Fetch Address Generation



18-740: Modern Computer Architecture & Design

Lecture 3: "Advanced Branch Prediction Techniques"

John Paul Shen
September 1, 2026

Next Time ...

- Assignments for This Week:
  - **Lab 1** will be posted by Tuesday (9/1)
- Required Reading Assignment:
  - **Chapters 5 and 9 of Shen and Lipasti (SnL)**
- Recommended Reference:
  - T. Yeh and Y. Patt, "Two-Level Adaptive Branch Prediction," Intl. Symposium on Microarchitecture, November 1991.

Electrical & Computer ENGINEERING

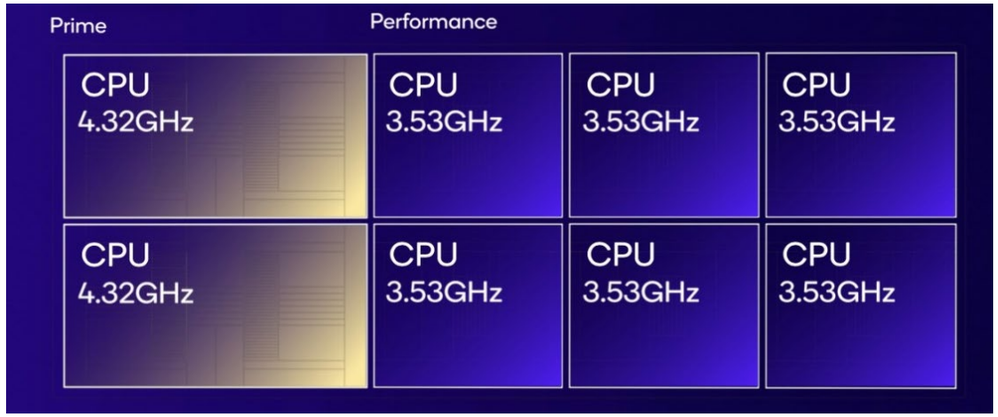
https://www.youtube.com/watch?v=hA_IrCYyyE4

Recent Qualcomm Snapdragon Mobile SoCs



https://www.msn.com/en-us/technology/hardware-and-devices/why-mobile-gamers-are-calling-this-the-best-snapdragon-yet/vi-AA1RHZ8Z?ocid=socialshare





	Snapdragon 8 Elite	Snapdragon 8 Gen 3	Snapdragon 8 Gen 2
CPU Cluster Configuration	Oryon CPU Cores: 2x 4.32GHz (Phoenix-L) 6x 3.53GHz (Phoenix-M)	1x 3.3GHz (Cortex-X4) 3x 3.2GHz (Cortex-A720) 2x 3GHz (Cortex-A720) 2x 2.3GHz (Cortex-A520 Refresh)	1x 3.19GHz (Cortex-X3) 2x 2.8GHz (Cortex-A715) 2x 2.8GHz (Cortex-A710) 3x 2.0GHz (Cortex-A510)
GPU	Adreno (ray tracing support)	Adreno (ray tracing support)	Adreno 740 (ray tracing support)
NPU	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support	Hexagon (fused scalar, tensor, and vector) Mixed precision INT8/INT16 INT4 support
RAM support	LPDDR5X	LPDDR5X	LPDDR5X
Process	TSMC 3nm	TSMC 4nm (N4P)	TSMC 4nm (N4)

Carnegie Mellon University
Electrical & Computer Engineering

New Silicon Initiative

Interested in chip design, circuit design, or computer architecture?

Join our special event featuring Apple and CMU ECE faculty discussing career opportunities in these exciting fields.

Afterwards, network with Apple engineers and CMU over burritos and boba.

New Silicon Initiative Fall Kickoff
Tuesday, September 1, 2026
4:00–6:00 p.m. ET (check-in begins at 3:00 p.m.)
McConomy Auditorium

This event is open to all CMU students, faculty, and staff. This slide is shared with the ECE department.

Scan or click the QR code to register

Questions?

nsi@apple.com

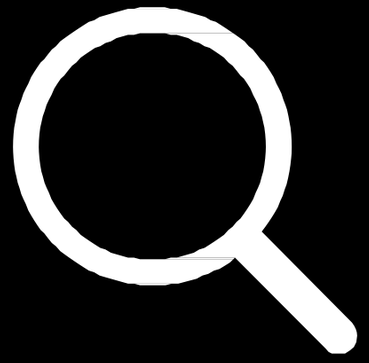
TM and © 2026 Apple Inc. All rights reserved.

Interested in VLSI, silicon engineering, and computer architecture?

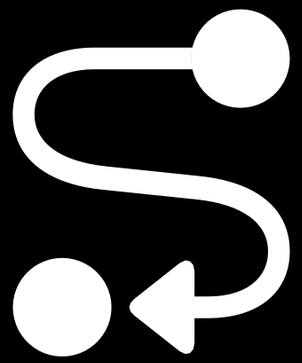
Have a 1:1 Flash Chat with an Apple engineer!



Get advice on your coursework, career, and internship prep



Discover open roles at Apple that may be a fit for you



Learn about engineer career paths that led to Apple

Sign up here!

https://s.apple.com/dI7J5r5UM7

TM and © 2026 Apple Inc. All rights reserved.